

Week 13 • Refactoring & owning the code

Technical Interaction Design • ITU • Autumn 2026

Built 18 September 2026 • commit 4ef64e0

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-13.pdf>

CONTENTS

Component Extraction & Semantic React Architecture

- Universal Principles (CSS-Agnostic)
- The Problem: "Div Soup" & "Class Soup"
- The Solution: Semantic Component Extraction
- The Four Core Principles
- Step-by-Step Tutorial: Component Extraction
- Advanced Pattern: Polymorphic Components
- Recommended File Structure
- Checklist: When to Extract a Component
- Practice Exercise
- Additional Resources
- Key Takeaways

Component Extraction & Semantic React Architecture

Source: [Lectures/Technical/Structure/Component-Extraction-Guide.md](#) — something unclear? [Open an issue](#), or send a pull request.

On the examples below. They are written in TypeScript with Tailwind class names, because they came from a different course. This course is JavaScript and MUI. Read them for the *decisions* — when a block deserves a name, when repetition earns a component — and ignore the syntax around them; nothing here depends on the stack.

Universal Principles (CSS-Agnostic)

These principles apply whether you use Tailwind, CSS Modules, styled-components, or plain CSS.

The Problem: “Div Soup” & “Class Soup”

Bad Example (Tailwind)

```
<div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
  <section className="flex w-full flex-col items-center gap-2 p-6">
    <div className="size-24 rounded-3xl bg-neutral-300" />
    <h2 className="text-2xl font-bold">{user.name}</h2>
  </section>
</div>
```

Bad Example (CSS Modules)

```
<div className={styles.outerContainer}>
  <section className={styles.innerSection}>
    <div className={styles.avatar} />
    <h2 className={styles.userName}>{user.name}</h2>
  </section>
</div>
```

Bad Example (Inline Styles)

```
<div style={{ margin: '0 auto', display: 'flex', width: '100%', maxWidth: '36rem',
flexDirection: 'column', gap: '1.5rem', padding: '1rem 1rem 1.5rem 1rem' }}>
  <section style={{ display: 'flex', width: '100%', flexDirection: 'column', alignItems:
'center', gap: '0.5rem', padding: '1.5rem' }}>
    <div style={{ width: '6rem', height: '6rem', borderRadius: '1.5rem', background:
'#d4d4d4' }} />
    <h2 style={{ fontSize: '1.5rem', fontWeight: 'bold' }}>{user.name}</h2>
  </section>
</div>
```

What's the problem in ALL these examples?

- Can't tell what these divs/sections represent
 - Styling details overwhelm structure
 - Not reusable
 - Hard to review without CSS knowledge
-

The Solution: Semantic Component Extraction

Good Example (Works with ANY CSS approach)

```
<ProfileContainer>
  <ProfileHeader>
    <Avatar user={user} />
    <UserName>{user.name}</UserName>
  </ProfileHeader>
</ProfileContainer>
```

Benefits:

- Clear component hierarchy
 - Communicates intent immediately
 - Backend devs can understand structure
 - Reusable across the app
 - Styling details hidden in component files
-

The Four Core Principles

1. The Tech Lead Test

"If someone who doesn't know your CSS framework can't understand your component hierarchy, you need better abstraction."

Ask yourself: Could a backend developer review this code and understand the page structure?

2. The Semantic Naming Test

"If you can describe a component's purpose in 1-2 words, that should be its name."

```
// Bad: What is this?
<div className="...">

// Good: Immediately clear
<ProfileCard>
<NavigationMenu>
<PricingTable>
```

3. The DRY Test (Don't Repeat Yourself)

"If you copy-paste the same styling pattern more than twice, extract a component."

4. The Single Level of Abstraction (SLA)

"Keep all JSX at the same conceptual level. Don't mix high-level components with low-level HTML details."

Bad (mixed abstraction levels):

```
<ProfilePage>
  <Header />  {/* High-level component */}
  <div className="flex gap-4">  {/* Low-level HTML */}
    <div className="w-1/3 bg-gray-100 p-4">  {/* Low-level HTML */}
      <p className="font-bold">Name</p>
      <p>{user.name}</p>
    </div>
  </div>
  <Footer />  {/* High-level component */}
</ProfilePage>
```

Good (consistent abstraction level):

```
<ProfilePage>
  <Header />
  <ProfileContent>
    <ProfileSection title="Name" value={user.name} />
    <ProfileSection title="Email" value={user.email} />
  </ProfileContent>
  <Footer />
</ProfilePage>
```

Step-by-Step Tutorial: Component Extraction

Example: Refactoring a Profile Page

Step 1: Identify the Problem

Original Code (275 lines, Profile.tsx):

```
const Profile = () => {
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
      <section className="flex w-full flex-col items-center gap-2 p-6">
        <div className="size-24 rounded-3xl bg-neutral-300">
          <img src={user.avatarUrl} alt={user.name} />
        </div>
        <h2 className="text-2xl font-bold">{user.name}</h2>
        <div className="flex items-center">
          {[...Array(5)].map((_, i) => (
            <Star key={i} className="h-5 w-5 text-yellow-400" />
          ))}
        </div>
        <div className="flex items-center gap-4 text-sm text-gray-600">
          <span className="flex items-center gap-1.5">
            <Briefcase className="h-4 w-4" /> {user.role}
          </span>
          <span>• Joined {formatDate(user.joinedDate)}</span>
        </div>
        <div className="flex gap-2">
          <button className="rounded-full bg-neutral-200 px-4 py-2">Email</button>
          <button className="rounded-full bg-neutral-200 px-4 py-2">Message</button>
          <button className="rounded-full bg-neutral-200 px-4 py-2">Call</button>
        </div>
      </section>
      { /* ... 200+ more lines ... */ }
    </div>
  );
};
```

Problems:

1. Everything in one file (275 lines)
 2. No semantic component names
 3. Can't understand structure without reading every line
 4. Not reusable
 5. Hard to test
-

Step 2: Identify Repeated Patterns

Ask yourself:

- What sections repeat?
- What could be reused elsewhere?
- What are the conceptual "things" on this page?

Pattern identification:

```
Profile Page
├── Container (layout wrapper)
├── Header
│   ├── Avatar
│   ├── Name
│   ├── Rating
│   ├── Metadata (role, joined date)
│   └── Action Buttons
├── About Section
├── Experience Section
│   └── Experience Entry (repeated)
├── Qualifications Section
├── Skills Section
└── Feedback Section
```

Step 3: Extract Layout Components

Create: `components/layout/ProfileContainer.tsx`

```
import { ReactNode } from 'react';

interface ProfileContainerProps {
  children: ReactNode;
}

export const ProfileContainer = ({ children }: ProfileContainerProps) => {
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
      {children}
    </div>
  );
};
```

OR with CSS Modules:

```
// ProfileContainer.module.css
.container {
  margin: 0 auto;
  display: flex;
  width: 100%;
  max-width: 36rem;
  flex-direction: column;
  gap: 1.5rem;
  padding: 1rem;
}

// ProfileContainer.tsx
import styles from './ProfileContainer.module.css';

export const ProfileContainer = ({ children }) => {
  return <div className={styles.container}>{children}</div>;
};
```

Step 4: Extract Domain Components

Create: `components/profile/ProfileHeader.tsx`

```
import { Avatar } from '@components/ui/Avatar';
import { StarRating } from '@components/ui/StarRating';
import { ProfileActions } from './ProfileActions';
import type { UserProfile } from '@types/user';

interface ProfileHeaderProps {
  user: UserProfile;
}

export const ProfileHeader = ({ user }: ProfileHeaderProps) => {
  return (
    <header className="flex w-full flex-col items-center gap-2 p-6">
      <Avatar src={user.avatarUrl} alt={user.name} size="large" />
      <h1 className="text-2xl font-bold">{user.name}</h1>
      <StarRating rating={user.rating} />
      <ProfileMetadata role={user.role} joinedDate={user.joinedDate} />
      <ProfileActions user={user} />
    </header>
  );
};
```

Benefits:

- Clear semantic name (`ProfileHeader`)
 - All header logic in one place
 - Reusable for similar profiles
 - Easy to test in isolation
-

Step 5: Extract Reusable UI Components

Create: **components/ui/Avatar.tsx**

```
interface AvatarProps {
  src: string;
  alt: string;
  size?: 'small' | 'medium' | 'large';
}

export const Avatar = ({ src, alt, size = 'medium' }: AvatarProps) => {
  const sizeClasses = {
    small: 'size-12 rounded-2xl',
    medium: 'size-16 rounded-2xl',
    large: 'size-24 rounded-3xl'
  };

  return (
    <div className={sizeClasses[size]}>
      <img src={src} alt={alt} className="h-full w-full object-cover" />
    </div>
  );
};
```

Now reusable everywhere:

```
<Avatar src={user.avatar} alt={user.name} size="large" />
<Avatar src={commenter.avatar} alt={commenter.name} size="small" />
```

Step 6: Extract Repeated List Items

Create: **components/profile/ExperienceEntry.tsx**

```

import { MapPin, Ship, CalendarDays } from 'lucide-react';
import type { UserExperience } from '@types/user';

interface ExperienceEntryProps {
  experience: UserExperience;
}

export const ExperienceEntry = ({ experience }: ExperienceEntryProps) => {
  return (
    <article className="flex items-center gap-4 border-b border-gray-100 py-4">
      <div className="size-24 rounded-3xl bg-neutral-300" />
      <div className="flex flex-col gap-1.5">
        <h4 className="font-semibold">{experience.title}</h4>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <MapPin className="h-4 w-4" />
          {experience.location}
        </p>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <Ship className="h-4 w-4" />
          {experience.vessel}
        </p>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <CalendarDays className="h-4 w-4" />
          {format(new Date(experience.date), "do MMM yyyy")}
        </p>
      </div>
    </article>
  );
};

```

Step 7: Assemble with Semantic Structure

Final: `pages/Profile.tsx`

```

import { ProfileContainer } from '@components/layout/ProfileContainer';
import { ProfileHeader } from '@components/profile/ProfileHeader';
import { ProfileSection } from '@components/profile/ProfileSection';
import { ExperienceList } from '@components/profile/ExperienceList';
import { QualificationsList } from '@components/profile/QualificationsList';
import { FeedbackList } from '@components/profile/FeedbackList';
import { useUserProfile } from '@hooks/useUserProfile';

export const Profile = () => {
  const { user, loading, error } = useUserProfile();

  if (loading) return <LoadingSpinner />;
  if (error) return <ErrorMessage error={error} />;
  if (!user) return <NotFound />;

  return (
    <ProfileContainer>
      <ProfileHeader user={user} />

      <ProfileSection title="About & Hobbies">
        <p>{user.about}</p>
      </ProfileSection>

      <ProfileSection title="Experience Log">
        <ExperienceList experiences={user.experiences} />
      </ProfileSection>

      <ProfileSection title="Qualifications">
        <QualificationsList qualifications={user.qualifications} />
      </ProfileSection>

      <ProfileSection title="Skills">
        <p>{user.skills}</p>
      </ProfileSection>

      <ProfileSection title="Feedback">
        <FeedbackList feedback={user.feedback} />
      </ProfileSection>
    </ProfileContainer>
  );
};

```

Results:

- **60 lines** instead of 275 (78% reduction)
 - **Immediately understandable** structure
 - **Each component** < 100 lines
 - **Reusable** components
 - **Testable** in isolation
 - **Reviewable** by anyone
-

Advanced Pattern: Polymorphic Components

Allow components to render different HTML elements for semantic correctness:

```
interface CardProps {
  children: ReactNode;
  element?: 'div' | 'article' | 'section' | 'aside';
}

export const Card = ({ children, element: Element = 'div' }: CardProps) => {
  return (
    <Element className="rounded-lg border p-4 shadow-sm">
      {children}
    </Element>
  );
};

// Usage - semantically correct everywhere
<Card element="article">
  <BlogPost />
</Card>

<Card element="aside">
  <Sidebar />
</Card>
```

Recommended File Structure

```
src/
├── components/
│   ├── layout/                # Layout containers
│   │   ├── ProfileContainer.tsx
│   │   ├── PageContainer.tsx
│   │   └── ContentWrapper.tsx
│   ├── ui/                   # Reusable UI primitives
│   │   ├── Avatar.tsx
│   │   ├── Button.tsx
│   │   ├── StarRating.tsx
│   │   └── Card.tsx
│   └── profile/              # Domain-specific components
│       ├── ProfileHeader.tsx
│       ├── ProfileSection.tsx
│       ├── ExperienceEntry.tsx
│       └── ProfileActions.tsx
└── pages/
    └── Profile.tsx           # Page composition
```

Checklist: When to Extract a Component

Extract when:

- ☐ A pattern appears 3+ times
- ☐ A section has clear responsibility (header, card, list item)
- ☐ JSX block exceeds 20-30 lines
- ☐ You can name it in 1-2 words
- ☐ It could be reused elsewhere
- ☐ It helps with Single Level of Abstraction

Don't extract when:

- ☐ It's only used once and simple (< 10 lines)
 - ☐ The abstraction makes code harder to understand
 - ☐ It requires too many props (> 8)
-

Practice Exercise

Given this code, extract semantic components:

```
const JobPage = () => {
  return (
    <div className="px-24 py-6">
      <div className="flex flex-col gap-4">
        <div className="flex flex-row gap-2.5">
          <div className="relative size-24 rounded-3xl bg-neutral-300">
            <Heart className="absolute top-2.5 right-2.5 cursor-pointer" />
          </div>
          <div className="flex flex-col justify-between py-2">
            <h2 className="font-semibold">{job.title}</h2>
            <div className="flex items-center gap-x-4">
              <p className="flex items-center gap-1">
                <Clock className="h-3.5 w-3.5" /> {job.type}
              </p>
              <p className="flex items-center gap-1">
                <CalendarDays className="h-3.5 w-3.5" /> {job.date}
              </p>
            </div>
          </div>
        </div>
        <div className="ml-auto flex flex-col gap-4">
          <button className="rounded-xl bg-neutral-200">Share</button>
          <button className="rounded-xl bg-green-500">Apply</button>
        </div>
      </div>
    </div>
  );
};
```

Your task:

1. Identify semantic sections
2. Create component names
3. Extract components
4. Refactor the page

Target structure:

```
<JobPageContainer>
  <JobHeader job={job} />
</JobPageContainer>
```

Additional Resources

Articles

- [Semantic JSX by Max Böck](#)
- [React Component Design Principles](#)
- [Single Level of Abstraction Principle](#)

Best Practices

- Favor composition over configuration
 - Use semantic HTML elements
 - Keep components focused (Single Responsibility)
 - Name components by their purpose, not appearance
 - Make components testable and reusable
-

Key Takeaways

1. **Component names should communicate intent** - Not implementation details
2. **Structure over styling** - Code should be readable regardless of CSS approach
3. **One level of abstraction** - Don't mix high and low-level code
4. **DRY principle** - Extract repeated patterns
5. **Think in components** - Not in divs, styling, or CSS classes

Remember: The goal is readable, maintainable, reviewable code that anyone on the team can understand.

Something unclear in this section? Open an issue.